

P1277R0

Subscripts On Parade

Published Proposal, 2018-10-07

This version:

<https://wg21.link/p1277r0>

Author:

[Isabella Muerte](#)

Audience:

EWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Current Render:

[P1277R0](#)

Current Source:

[slurps-mad-rips/papers/proposals/subscript-parade.bs](https://ericniebler.com/2018/10/07/subscript-parade/)

Abstract

Subscripting into a type that models a multidimensional array should be viable and possible without having to implement proxy types or using the call operator as an overload.

Table of Contents

1 Revision History

1.1 Revision 0

2 Motivation

3 Design

4 Wording

References

Informative References

§ 1. Revision History

§ 1.1. Revision 0

Initial Release 🎉

§ 2. Motivation

As multidimensional types such as `std::mdspan` and other HPC types come about, it is becoming increasingly clear that the recommended convention of simply overloading `operator ()` to pass in multiple indices is not enough. This paper seeks to permit adding multiple arguments to `operator []` to permit expressions like `mdspan[1, 2, 3]`. This change is compatible with existing code as overloads for these multiple arguments need to be present on a given type for this to work in the first place.

This paper works alongside [\[P1161\]](#), but takes the more bold statement that we need multiple arguments to the subscript operator *now* rather than later. After all, it has to start somewhere. It has to start sometime. What better place than here? What better time than *now*?

§ 3. Design

The current design of this paper is to permit multiple arguments to be passed to `operator []`. This changes the proposed behavior seen in [\[P1161\]](#), where `array[x, y]` is in fact not deprecated, but instead attempts to call `array.operator [] (x, y)`. This gives us the opportunity to have multidimensional array types that act like multidimensional arrays. This also saves us the pain of having to implement proxy types that are returned for the sole purpose of being indexed upon, or having to use `operator ()`, which while an existing convention is confusing when compared to other callable interfaces (that is, `mdspan` is not a Callable, yet it satisfies the interface for it). This also means that the built-in comma operator is not permitted inside of subscript expressions.

§ 4. Wording

In 7.6.19 Comma operator [**expr.comma**] add a paragraph

³The built-in comma operator shall not be used within the subscript operator.

In 11.5.5 Subscripting [**over.sub**]

¹ **operator** [] shall be a non-static member function with **exactly one parameter** **an arbitrary number of parameters**. It implements the subscripting syntax:

postfix-expression [*expr-or-braced-init-list* *expression-list*_{opt}]

Thus, a subscripting expression ~~*x*[*y*]~~ is interpreted as ~~*x.operator*[(*y*)]~~ for a class object ~~*x*~~ of type ~~*T*~~ if ~~*T::operator*[(*T1*)]~~ ~~*x*[*arg1*,...]~~ is interpreted as ~~*x.operator*[(*arg1*,...)]~~ for a class object ~~*x*~~ of type ~~*T*~~ if ~~*T::operator*[(*T1*, *T2*, *T3*)]~~ exists and if the operator is selected as the best match function by the overload resolution mechanism [*Example*:

```
struct X {  
  Z operator [] (std::initializer_list<int>);  
};  
X x;  
x[{1, 2, 3}] = 7; // OK: meaning x.operator[]({1, 2, 3})  
int a[10];  
a[{1, 2, 3}] = 7; // error: built-in subscript operator
```

```
struct X {  
  Z operator [] (std::initializer_list<int>);  
  Z operator [] (int x, int y, int z);  
};  
X x;  
x[{1, 2, 3}] = 7; // OK: meaning x.operator[]({1, 2, 3})  
x[1, 2, 3] = 7; // OK: meaning x.operator[](1, 2, 3)  
int a[10];  
a[{1, 2, 3}] = 7; // error: built-in subscript operator  
a[1, 2, 3] = 7; // error: built-in subscript operator
```

— *end example*]

²In subscript expressions, the built-in comma operator shall not be used

§ References

§ Informative References

[P1161]

Corentin Jabot. [Deprecate uses of the comma operator in subscripting expressions.](https://wg21.link/p1161r0) URL:
<https://wg21.link/p1161r0>